

📁 OOAD Ch15: Decorator、Factory 與 MVC (設計模式)

主題重點： CH15 延續 CH14 的 Design Patterns，聚焦三個實務上非常常見的設計主題：

1. 用 **Decorator Pattern** 在不修改既有類別的前提下，動態替物件加責任。
2. 用 **Factory Pattern / Factory Method / Abstract Factory** 封裝 *new* 與具體類別依賴，讓建立物件的變動被隔離。
3. 用 **MVC** 把 GUI / 使用者事件 / 核心運算邏輯分離，避免 UI 改動拖垮整個系統。

本筆記依照要求：不要省略任何內容。下方包含整理後的導讀、模式對照、觀念補強，以及使用者提供原文的完整課堂筆記內容。

👤 Source Materials

- 來源：使用者提供的 `ch15_note.txt`。
- 原始檔備份：[ch15_note.txt](#)
- 內容說明：涵蓋老師上課音檔內容，以及三份簡報：Decorator、Factory、MVC。
- 整理原則：保留所有老師例子、術語、警告與設計原則；不過度精簡。

📖 0. 本章總覽

本章可以理解成三個「常見變動來源」的處理方式：

1. **功能組合會變動** → 用 **Decorator Pattern**
 - 例子：Starbuzz 咖啡 + 牛奶 / 豆漿 / 摩卡 / 奶霜。
 - 問題：如果每一種組合都寫一個子類別，會發生 **Class Explosion (類別爆炸)**。
 - 解法：用裝飾者一層一層 wrap 核心物件，執行時期動態加責任。
2. **建立哪個具體物件會變動** → 用 **Factory Pattern**
 - 例子：披薩店依照口味與地區產生不同 pizza。
 - 問題：核心程式碼到處寫 `new ConcreteClass()`，會依賴具體類別，違反 OCP / DIP。
 - 解法：把創建物件的部分抽出來，由 Simple Factory、Factory Method 或 Abstract Factory 管理。
3. **UI / 使用者操作方式會變動** → 用 **MVC**
 - 例子：計算機程式若把加減乘除寫在 GUI EventHandler 裡，UI 一改整個程式就要改。
 - 問題：Computation logic 與 GUI 混在一起。
 - 解法：Model 管核心資料與運算，View 管呈現，Controller 管使用者事件與操作解讀。

💡 1. 本章核心設計原則

1.1 Open-Closed Principle (OCP, 開放封閉原則)

Classes should be open for extension, but closed for modification.

意思是：

- 當系統需要新增功能時，應該能透過「擴充」完成。
- 不應該頻繁打開已經寫好、測試好的舊 source code 來修改。
- 不是整個系統每個地方都必須完美符合 OCP，而是要把心力放在「最有可能變動」的區域。

老師的硬體比喻：

- 早期 PC 擴充要拆機殼、插介面卡：對修改開放，不理想。
- USB 隨插即用：對擴充開放、對修改封閉，比較符合 OCP。

1.2 Dependency Inversion Principle (DIP, 依賴反轉原則)

高階元件不應該依賴低階元件；兩者都應該依賴抽象。不要依賴具體類別。

在 Factory 章節中：

- `PizzaStore` 是高階元件。
- `CheesePizza`、`PepperoniPizza` 等具體 pizza 是低階元件。
- 如果 `PizzaStore` 直接 `new CheesePizza()`，高階元件就依賴低階具體類別。
- 透過 factory，兩邊都依賴抽象 `Pizza`，依賴方向被反轉。

老師補充的三大理想方針：

1. 沒有變數應該持有具體類別的 reference。
2. 沒有類別應該繼承具體類別。
3. 沒有方法應該 override 基礎類別已實作的方法。

實務提醒：這是理想目標，不是每一行程式都要做到；重點是變動核心區域要盡量維持彈性。

✂️ 2. 三大主題快速對照

主題	核心問題	典型壞味道	解法	關鍵句
Decorator	功能組合太多	每種組合都做一個 subclass，造成 class explosion	用 decorator wrap component，動態加責任	繼承是為了 type matching，不是為了繼承行為
Factory Method	建立物件依賴具體類別	核心流程到處寫 <i>new</i> 與 <i>if-else</i>	把創建延遲到 subclass 的 factory method	Defer instantiation to subclasses
Abstract Factory	一整組產品家族需要一致	原料 / 產品組合被寫死或混搭錯誤	用 factory interface 產生 related / dependent object families	用 composition 建立產品家族

主題	核心問題	典型壞味道	解法	關鍵句
MVC	UI 與運算邏輯混雜	EventHandler 裡直接寫 computation logic	Model / View / Controller 分離	Model 不可以含 GUI 指令

🔑 3. 考試與作業容易問的重點

3.1 Decorator Pattern 必背點

- 解決類別爆炸。
- 讓功能可以在 runtime 動態組合。
- Concrete Decorator 和 Concrete Component 都必須有相同的抽象型別，例如都繼承 / 實作 `Beverage`。
- 這裡用繼承不是為了繼承行為，而是為了 **type matching**。
- 真正的行為擴充來自 object composition 與 delegation。
- Java I/O 是經典實例：`FileInputStream` + `BufferedInputStream` + `FilterInputStream`。

3.2 Factory Pattern 必背點

- `new` 代表依賴 concrete class。
- Simple Factory 不是正式 design pattern，只是 programming idiom。
- Factory Method：用 inheritance，讓 subclass 決定建立哪個產品。
- Abstract Factory：用 object composition，建立 related / dependent product families。
- Factory 的目標是把變動的物件建立邏輯從穩定流程中抽離出來。

3.3 MVC 必背點

- Model：business logic / computation logic / state / data。
- View：render model data，負責畫面呈現。
- Controller：handle user events，解讀操作，呼叫 Model API 或要求 View 改變顯示狀態。
- Model 最大鐵則：不能包含 GUI 相關程式碼。
- MVC 中包含三個模式：
 - Model-View：Observer Pattern。
 - View-Controller：Strategy Pattern。
 - View 自身：Composite Pattern。

📖 4. 逐段完整課堂筆記（保留原始內容，不省略）

以下完整保留提供的 CH15 課堂筆記內容，並只做 Obsidian 化標題與連結整理。

第一部分：裝飾者模式 (Decorator Pattern)

老師以知名的連鎖咖啡店 Starbuzz (星巴克) 的點餐系統作為切入點，說明當系統面臨大量組合需求時所遭遇的問題，進而引出裝飾者模式。

1. 遭遇的問題：類別爆炸 (Class Explosion)

- 初始設計**：一開始設計了一個抽象的 `Beverage` 父類別，所有的咖啡種類（如 HouseBlend, DarkRoast, Espresso, Decaf）都繼承它，並各自實作 `cost()` 方法來計算價格。
- 現實狀況**：顧客買咖啡會要求加料（如蒸氣牛奶、豆漿、摩卡/巧克力、奶霜）。如果針對每一種「咖啡豆 + 配料」的組合都寫一個繼承的類別（例如：`HouseBlendWithSteamedMilkAndMocha`），會產生驚人的組合數量，導致**類別爆炸 (Class Explosion)**，這顯然是非常愚蠢的做法。
- 直覺但有缺陷的解法**：如果在父類別 `Beverage` 中加入表示是否加了牛奶、摩卡等配料的布林值 (Boolean) 與對應的 getter/setter，並在父類別的 `cost()` 裡先算好配料的錢，再讓子類別透過 `super.cost()` 加上咖啡豆的錢，看似解決了問題。
- 缺陷分析**：這種做法違背了系統維護的彈性。當今天配料價格改變（例如奶霜漲價），或者推出新配料時，我們都必須把父類別 (`Beverage`) 的程式碼打開來修改。甚至當推出新飲料（例如冰茶 Iced Tea）時，這些針對咖啡的配料並不適用，但冰茶卻無奈地繼承了這些不合理的屬性與方法。這在系統設計上是個嚴重的問題。

2. 核心設計原則：開放封閉原則 (Open-Closed Principle)

- 定義**：**類別應該對擴充開放，對修改封閉 (Classes should be open for extension, but closed for modification)**。意思是，當系統需要新增功能時，你應該能夠直接進行擴充，而不需要把舊有的、已經測試好的原始碼 (Source code) 打開來修改。
- 硬體歷史比喻**：老師舉了 20 年前 PC 電腦擴充的歷史為例。早期要擴充硬體，必須拿螺絲起子拆開機殼 (Open case to install interface card)，這就是「對修改開放」，違背了原則。而後來的 USB 介面做到了真正的隨插即用 (Open for extension)，不必再拆機殼 (Close for modification)。
- 開發建議**：要讓所有系統完全符合 OCP 原則需要耗費大量的時間與心力，因此只要將心力花在「最有可能發生變動」的區域即可，這需要靠長期的物件導向設計經驗來判斷。

3. 裝飾者模式定義與運作方式

- 定義**：動態地將額外的責任附加到物件上。若要擴充功能，裝飾者提供了比繼承更有彈性的替代方案。
- 運作邏輯 (Wrap)**：用一個裝飾者物件 (Decorator) 將被裝飾的物件 (Component) 包裝起來。如果客人要一杯 Dark Roast 加 Mocha 跟 Whip，我們會先實體化 Dark Roast 物件，然後把 Mocha 包在它外面，最後再把 Whip 包在最外面。
- 委派 (Delegation)**：當最後計算總價時，我們呼叫最外層 (Whip) 的 `cost()` 方法，它會去呼叫內層 (Mocha) 的 `cost()`，再往內呼叫 (Dark Roast) 的 `cost()`，拿到核心價格後，層層將配料價格往上加，最後回傳總價。
- 型別相符 (Type Matching)**：裝飾者必須與被裝飾者擁有一樣的介面/父類別（例如 `Mocha` 與 `DarkRoast` 都是 `Beverage`）。這裡使用繼承是為了達到「型別相符」，而不是為了繼承行為，行為是透過物件的組合 (Composition) 在執行時期 (Runtime) 動態獲得的。

4. 業界實際範例：Java I/O API

- 許多初學者一開始看不懂 Java 的 I/O 系統，其實它就是標準的裝飾者模式。
- `InputStream` 是一個抽象的元件 (Component)。`FileInputStream` 是具體的基底元件，只負責讀檔。
- `FilterInputStream` 是所有裝飾者的抽象父類別。底下的 `BufferedInputStream` (加入緩衝功能) 或 `LineNumberInputStream` (加入計算行號功能) 都是具體裝飾者。
- 使用方式：你會先 `new FileInputStream("test.txt")`，再用 `BufferedInputStream` 將其包起來，甚至可以自己寫一個裝飾者（如老師展示將字元轉為小寫的 `LowerCaseInputStream`），透過 `super.read()` 攔截位元組並進行轉換，展現了無比的擴充威力。

第二部分：工廠模式 (Factory Pattern)

這個章節在探討建立物件時，如何避免將程式碼寫死，維持系統的彈性。

1. 遭遇的問題：依賴具體類別與 `new` 關鍵字

- Program to implementation 的危害**：當我們寫下 `new` 這個關鍵字時，後面跟著的一定是一個「具體 (concrete)」的類別。如果核心程式碼裡面充滿了這類依賴具體類別的邏輯，系統將變得非常脆弱。

- 不良案例：假設你的程式包含判斷不同模式的邏輯，例如「野餐模式就 new 綠頭鴨、打獵模式就 new 誘餌鴨」；或者「當鬧鐘響起就 new 1000 隻煩人的狗，生日到就 new 會唱歌的狗」。未來只要有新的類別加入或修改，你就必須把這段核心程式碼「挖出來」重新檢視並修改，違背了開放封閉原則，也容易衍生 Bug。

2. 披薩店範例與簡單工廠 (Simple Factory)

- 範例背景：一個 `orderPizza(String type)` 的方法中，包含了根據 `type` ("cheese", "pepperoni", 等) 來 new 不同披薩的 `if-else` 邏輯，緊接著進行 `prepare()`, `bake()`, `cut()`, `box()` 等標準動作。
- 分離變動：準備、烘烤等步驟是不變的，而 `if-else` new 披薩種類的部分是隨著新口味不斷變動的。我們應該將變動的部分抽離出來。
- Simple Factory (簡單工廠)：將物件的創建邏輯封裝到一個名為 `SimplePizzaFactory` 的類別，並透過如 `createPizza()` 這個靜態方法 (Static Method) 來呼叫。
- 老師特別強調：Simple Factory 其實不是一個正式的設計模式，而是一種常見的寫程式習慣 (Programming Idiom)。

3. 工廠方法模式 (Factory Method Pattern)

- 定義：定義一個用來建立物件的介面，但讓子類別 (subclass) 決定要實體化哪一個類別。工廠方法將實體化的步驟推遲 (defer) 到次類別。
- 架構改良：將 `PizzaStore` 變成一個抽象父類別 (Creator)，裡面擁有抽象方法 `abstract createPizza()`。
- 子類別決定：建立諸如 `NYStylePizzaStore` (紐約店) 或 `ChicagoStylePizzaStore` (芝加哥店) 來繼承 `PizzaStore`，並由這些具體的子類別去 Override (覆寫) `createPizza()` 方法，決定產生紐約風味或是芝加哥風味的披薩。這樣 `PizzaStore` 的核心邏輯就與具體被創建的披薩完全解耦 (Decoupled) 了。

4. 依賴反轉原則 (Dependency Inversion Principle, DIP)

- 定義：高階元件不應該依賴低階元件，兩者都應該依賴於抽象 (Abstractions)。不要依賴具體類別。
- 原本 `PizzaStore` (高階元件) 會直接依賴各種具體的 `CheesePizza` (低階元件)。透過工廠模式，高階元件和低階元件現在都依賴於抽象的 `Pizza` 介面，依賴關係被成功反轉。
- 避免違反 DIP 的三大方針：
 - 沒有變數應該持有一個對具體類別的參考 (Reference)。
 - 沒有任何類別應該繼承自一個具體類別。
 - 沒有任何方法應該覆寫其基礎類別已經實作的方法。
(老師補充這是一個理想目標，實務上盡量將變動的核心區域維持這樣即可。)

5. 抽象工廠模式 (Abstract Factory Pattern)

- 定義：提供一個介面，用於建立相關或相依物件的「家族 (families)」，而不需要明確指定具體類別。
- 應用場景：當你需要一組原物料，比如紐約披薩需要特製麵團、大蒜醬料、雷吉亞諾起司；芝加哥披薩需要厚餅皮、番茄醬料、莫札瑞拉起司。透過 `PizzaIngredientFactory` 介面，實作不同區域的具體工廠，來確保生產出來的元件是一致的組合。
- 兩者差異：工廠方法 (Factory Method) 仰賴繼承 (Inheritance) 來委派創建；抽象工廠 (Abstract Factory) 仰賴物件組合 (Object Composition) 來建立一整個系列的產品。

第三部分：MVC 架構 (Model-View-Controller Paradigm)

這部分探討如何將使用者介面 (GUI) 與業務邏輯乾淨地分離，MVC 可能是人類最早使用的幾個架構/模式之一。

1. MVC 的動機與反面教材

- 失敗的計算機面試題：老師舉了對岸面試的情境，請應徵者寫一個簡單的計算機。應徵者用了 C# 直覺地把所有的運算邏輯寫在 GUI 的事件處理器 (EventHandler) 裡（例如在按鈕點擊事件裡直接讀取 TextBox 並進行加法）。
- 致命缺陷：計算邏輯 (Computation logic) 跟 UI 程式碼嚴重混雜。UI 依賴運算邏輯，運算邏輯也依賴 UI。
- GUI 的不穩定性：GUI 是軟體中最不穩定的元件。設計師或老闆可能會要求把按鈕改成 Console 介面、不用滑鼠改用鍵盤輸入、為了 Usability Test (可用性測試) 改變操作習慣，或是把系統移植到 Mac、Web 或行動裝置。如果照直覺寫，每次 UI 一改，整支程式都得打開來大修，違反了「開放封閉原則」。

2. MVC 架構的核心元件

為了解決這問題，必須將程式碼分為三大區塊：

- Model (模型)：**
 - 負責核心運算邏輯 (Computation model / business logic handler)、管理應用程式狀態和所有資料。
 - 最大鐵則：**Model 的程式碼內絕對不能包含任何一行與 GUI (如 TextBox, Button) 相關的指令，Model 必須完全獨立於 View 和 Controller。
 - 當 Model 的狀態被改變時，它可以發送通知給那些對於狀態有興趣的觀察者。
- View (視圖)：**
 - 負責將 Model 的資料彩現 (Render) 到視覺化物件上，負責在螢幕上畫出介面給使用者看。
 - 通常包含視窗、按鈕、文字框等元件。View 從 Model 取得資料以更新畫面。
- Controller (控制器)：**
 - 負責處理使用者的事件 (User events)。
 - 當使用者操作 View (如點擊滑鼠、敲擊鍵盤)，View 自身不處理邏輯，而是將事件丟給 Controller。
 - Controller 會解讀這些使用者動作的意義，並決定要呼叫 Model 的哪一個 API 來進行運算，或是要求 View 改變介面顯示狀態（例如啟用或停用某個按鈕）。

3. MVC 中運用到的設計模式 (Design Patterns in MVC)

MVC 其實是數個經典設計模式的結合體：

- Observer Pattern (觀察者模式)：**Model 與 View 之間的關係。Model 扮演可被觀察的主題，View 扮演觀察者。當 Model 內部資料改變時，會通知已註冊的 View 更新畫面。如此一來，Model 完全不必認識 View 的具體實作，達到解耦。老師舉例，就像你在寫 GUI 程式時加入 `addMouseListener(this)`，就是把觀察者註冊進去的動作。
- Strategy Pattern (策略模式)：**View 與 Controller 之間的關係。View 將處理操作的策略委派給 Controller。如果需要改變操作行為（例如在 3D 遊戲 Second Life 中，發現使用者不喜歡用滑鼠右鍵旋轉視角，想改用別的按鍵），你只需要把 Controller 替換掉，完全不必動到 View 的呈現程式碼。
- Composite Pattern (組合模式)：**View 自身的結構。View 的呈現往往由多個視窗、面板、標籤元件層層疊加組成，這是一個標準的組合模式。

4. 實務開發的特性與 Web MVC

- UI 開發工具特性：**現代的 GUI 開發環境大多配有設計工具 (Designer)，用拖拉的方式就會自動產生按鈕的程式碼。如果你在自動產生的 UI 事件區塊寫滿邏輯，一旦重新拖拉設計，舊程式碼很容易遺失或需要做無謂的 Copy/Paste。因此實務上，View 的觸發事件裡通常只寫「非常簡短的一行程式碼」，直接把狀態和參數 pass 交給 Controller 處理。
- MVC 的優勢：移植性：**當你需要將系統從 Windows 移植到 Mac 或 Web 時，如果架構遵從 MVC，你的 Model 可以直接打包搬過去不須重寫，只需要重新抽換撰寫符合新平台的 View 和 Controller 即可。
- Web MVC：**老師總結，雖然在撰寫小型本機 GUI 程式時，可能覺得切分 MVC 有點做作，但在開發 Web Application (如 JSP、PHP、ASP) 時，MVC 的架構是自然天成的。HTTP 請求進來由 Servlet 處理 (扮演 Controller)，呼叫後端資料庫或 JavaBean (扮演 Model)，最後由 JSP 產生 HTML 呈現 (扮演 View)。

✅ 5. 本章最後整理：老師想傳達的設計精神

CH15 的三個主題共同指向同一件事：

把會變的東西隔離起來，讓穩定的部分不要被牽連。

- Decorator：把「功能加料組合」的變動隔離成 decorator。
- Factory：把「具體物件建立」的變動隔離到 factory。
- MVC：把「UI 呈現與操作方式」的變動隔離在 View / Controller，讓 Model 保持乾淨。

因此本章不是只要背模式名稱，而是要看出每個模式背後共同 OO 思維：

- Program to an interface, not an implementation.
- Favor composition over inheritance.
- Encapsulate what varies.
- Open for extension, closed for modification.
- Depend upon abstractions, not concrete classes.